

Resources and Deadlocks

1 Introduction

Central Idea

A **resource** is anything a process must **acquire**, **use**, and **release**. Examples include devices, files, database records, memory, and protected software objects.

Deadlock becomes possible when a process holds one exclusive resource while waiting for another resource held by a different process.

Main Point

A small change in resource-acquisition order can change a program from deadlock-free to deadlock-prone.

2 What Counts as a Resource?

A computer normally has many kinds of resources. Some resources have only one instance, while others have several identical instances.

Resource type	Example	Meaning of acquisition
Hardware device	Printer, Blu-ray drive, disk unit	A process gets permission to use the device.
Data object	Database record, shared file	A process gets exclusive access before reading or updating it.
Memory	Main-memory pages or address space	A process occupies memory while it runs.
Software object	Lock, semaphore-protected object	A process enters a protected region.

Identical Instances

If several identical copies exist, any one copy can satisfy the request. For example, if three Blu-ray drives are free, a process requesting one drive can receive any of them.

3 Preemptable and Nonpreemptable Resources

Preemptable Resource

A **preemptable resource** can be taken away from its current owner without damaging the computation or causing failure.

Example: memory on a system with paging or swapping. The operating system can swap a process out, give memory to another process, and later restore the swapped-out process.

Nonpreemptable Resource

A **nonpreemptable resource** cannot be taken away from its current owner at an arbitrary moment without risking failure or corrupted output.

Example: a Blu-ray recorder while writing. Taking it away in the middle of writing may corrupt the disc.

Kind	Can be taken away?	Typical deadlock effect
Preemptable	Yes, if the system can save and restore state safely.	Potential deadlocks can often be resolved by reallocating the resource.
Nonpreemptable	No, not safely at an arbitrary time.	Real deadlocks usually involve these resources.

Context Matters

Preemptability depends on the system. Memory is preemptable on many PCs with paging, but may not be preemptable on a device without paging or swapping.

4 Standard Resource-Use Sequence

Resource Life Cycle

Request resource → Use resource → Release resource

If the resource is free, the process gets it. If it is busy, the process waits.

- Some systems block the process automatically.
- Some systems return an error, so the process sleeps and retries.
- A process that only retries cannot do useful work, so it is effectively blocked.

Simplifying assumption

For deadlock discussion, we usually assume that a process whose resource request is denied is put to sleep until the resource becomes available.

5 Resource Acquisition with Semaphores

When the operating system does not directly manage a resource, processes can protect it with a semaphore or mutex.

Semaphore Protection

For a binary semaphore initialized to 1, **down** acquires the resource and **up** releases it.

5.1 One Resource

```
typedef int semaphore;
semaphore resource_1 = 1;

void process_A(void) {
    down(&resource_1);    /* request resource 1 */
    use_resource_1();      /* use resource 1 */
    up(&resource_1);       /* release resource 1 */
}
```

Only one process can use `resource_1` at a time.

5.2 Two Resources

```
typedef int semaphore;
semaphore resource_1 = 1;
semaphore resource_2 = 1;

void process_A(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources();
    up(&resource_2);
    up(&resource_1);
}
```

Sequential acquisition is common. It becomes risky when different processes acquire the same resources in different orders.

6 Deadlock-Free Acquisition Order

Ordering Rule

If every process acquires shared resources in the same global order, circular waiting is avoided for those resources.

Both processes below request `resource_1` before `resource_2`. One process may wait, but no circular wait forms.

```
typedef int semaphore;
semaphore resource_1 = 1;
semaphore resource_2 = 1;

void process_A(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources();
    up(&resource_2);
    up(&resource_1);
}

void process_B(void) {
    down(&resource_1);
    down(&resource_2);
}
```

```
    use_both_resources();  
    up(&resource_2);  
    up(&resource_1);  
}
```

What happens at runtime?

If *A* gets `resource_1` first, *B* blocks before holding anything. *A* can get `resource_2`, finish, release both, and then *B* can run.

7 Potential Deadlock from Different Orders

Here *A* requests `resource_1` first, but *B* requests `resource_2` first.

```
typedef int semaphore;  
semaphore resource_1 = 1;  
semaphore resource_2 = 1;  
  
void process_A(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources();  
    up(&resource_2);  
    up(&resource_1);  
}  
  
void process_B(void) {  
    down(&resource_2);  
    down(&resource_1);  
    use_both_resources();  
    up(&resource_1);  
    up(&resource_2);  
}
```

Potential deadlock

This code does not deadlock on every run. It deadlocks only if *A* holds `resource_1` while *B* holds `resource_2`.

8 Worked Deadlock Trace

Problem

Processes *A* and *B* both need resources R_1 and R_2 . Process *A* requests R_1 then R_2 . Process *B* requests R_2 then R_1 . Show how deadlock can occur.

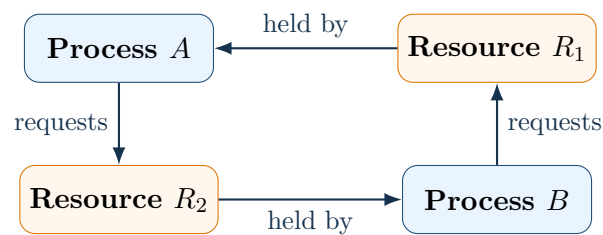
Solution

Step	Process A	Process B	State
1	Acquires R_1 .	Not yet running.	A holds R_1 .
2	Pauses before requesting R_2 .	Acquires R_2 .	A holds R_1 , B holds R_2 .
3	Requests R_2 .	Still holds R_2 .	A blocks because R_2 is unavailable.
4	Still blocked.	Requests R_1 .	B blocks because R_1 is unavailable.
5	Waiting for B to release R_2 .	Waiting for A to release R_1 .	Neither process can continue.

Answer

The system is deadlocked: A waits for B , and B waits for A . Neither can reach the release code.

9 Resource Allocation View



Circular Wait Pattern

A waits for R_2 , R_2 is held by B , B waits for R_1 , R_1 is held by A

This cycle is the deadlock. Because the resources are nonpreemptable, the operating system cannot simply take them away safely.